PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 05 -

GERENCIANDO AMBIENTES COM TERRAGRUNT

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 13
O QUE VOCÊ VIU NESTA AULA? 14
REFERÊNCIAS 15

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VEM POR AÍ?

Gerenciar infraestrutura em múltiplos ambientes é um dos maiores desafios para times que trabalham com IaC. Até aqui, vimos como o Terraform e o OpenTofu permitem criar recursos de forma modular e automatizada. Mas conforme os projetos crescem, surgem problemas de repetição de código, dificuldade em manter consistência entre ambientes e aumento do risco de erros humanos.

É justamente para resolver esses pontos que entra o **Terragrunt**. Nesta aula, vamos entender como ele se posiciona como uma camada extra sobre o Terraform, trazendo práticas de organização que evitam duplicação, centralizam configurações e simplificam o trabalho em ambientes como **dev, staging e prod**. Também veremos como o Terragrunt permite execuções em cascata, integra backends remotos e aplica boas práticas que ajudam times grandes a manter a infraestrutura sob controle.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

O Hands On desta aula vai mostrar, na prática, como montar uma estrutura organizada que evita repetição de código, permitindo que ambientes como **dev, staging e prod** compartilhem a mesma base e se diferenciem apenas no que é necessário.

Nos vídeos, veremos como criar diretórios hierárquicos onde cada ambiente possui seu próprio arquivo ***terragrunt.hcl***, herdando configurações comuns definidas em níveis superiores. Essa abordagem ilustra bem o princípio **DRY (Don't Repeat Yourself)**, mostrando como centralizar definições de backends, variáveis e provedores, enquanto cada ambiente herda e sobrescreve apenas o que precisa.

Também exploraremos o fluxo de trabalho com o comando ***terragrunt run-all,*** que permite executar planos ou aplicações em cascata, abrangendo múltiplos módulos de uma só vez. Esse recurso será útil para observar como ambientes inteiros podem ser provisionados de forma coordenada, com menos esforço e maior consistência. O objetivo aqui é preparar a base para que times consigam gerenciar grandes projetos de IaC com clareza, escalabilidade e governança.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

Até aqui usamos o Terraform para criar módulos e automatizar recursos. Mas quando surge a necessidade de trabalhar com **múltiplos ambientes (dev, staging, prod)**, o código começa a se repetir: o mesmo backend, os mesmos providers, as mesmas variáveis copiadas em várias pastas. Isso gera **duplicação, risco de inconsistência e mais esforço de manutenção**.

O **Terragrunt** nasce exatamente para resolver esse problema. Ele funciona como uma camada em cima do Terraform/OpenTofu, que adiciona três grandes funcionalidades:

- **Herança de configurações** → ambientes herdam definições comuns de arquivos pais.
- **Organização de diretórios** → cada ambiente só sobrescreve o que precisa.
- **Execução orquestrada** → com `run-all`, é possível aplicar módulos em ordem e respeitar dependências.

O que é Terragrunt e por que usar

No Terraform puro, o mesmo bloco de backend ou provider é duplicado em cada módulo. No Terragrunt, centralizamos isso em um único arquivo **pai** (terragrunt.hcl no diretório raiz), e todos os ambientes **herdam** essa configuração.

- Centraliza **configurações comuns** (remote state, provider, variáveis);
- Injeta **inputs** em módulos;
- Permite **herança** entre diretórios (um “pai” define regras; os “filhos” herdam e só mudam o necessário);
- Orquestra execuções com ***run-all*** (aplica/planeja módulos em cascata).

Exemplo: apontando para um módulo Terraform

```
    # infra/dev/us-east-1/vpc/terragrunt.hcl
    terraform {
      # Aponta para um módulo (local, git, registry). Pode ser
OpenTofu também.
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
  source = "../../../modules/vpc"
}

inputs = {
  name       = "dev-vpc"
  cidr_block = "10.10.0.0/16"
  azs        = ["us-east-1a","us-east-1b"]
}
```

Aqui, o Terragrunt não substitui o Terraform — ele apenas **chama o módulo** já existente e injeta as variáveis necessárias. O ganho está em como organizamos **vários desses módulos** sem repetir código.

Estrutura de diretórios, herança e DRY

Um repositório organizado com Terragrunt costuma separar **código reutilizável** (módulos) de **ambientes vivos** (live):

```
repo/
├─ modules/
│  ├─ vpc/
│  └─ eks/
└─ live/
   ├─ terragrunt.hcl          # raiz (herdado por todos)
   ├─ dev/
   │  └─ us-east-1/
   │     ├─ vpc/terragrunt.hcl
   │     └─ eks/terragrunt.hcl
   └─ prod/
      └─ us-east-1/
         ├─ vpc/terragrunt.hcl
         └─ eks/terragrunt.hcl
```

- `modules/` → contém o código puro do Terraform, reaproveitável.
- `live/` → contém os ambientes que realmente serão aplicados.

Arquivo raiz: configurações comuns a todos

```
# live/terragrunt.hcl
locals {
  aws_region   = "us-east-1"
  state_bucket = "iac-terraform-state"
  lock_table   = "terraform-locks"
}

remote_state {
  backend = "s3"
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
    config = {
      bucket         = local.state_bucket
      key                                                          =
"${path_relative_to_include()}/terraform.tfstate"
      region         = local.aws_region
      dynamodb_table = local.lock_table
      encrypt        = true
    }
  }

  generate "provider" {
    path      = "provider.tf"
    if_exists = "overwrite"
    contents  = <<EOF
  provider "aws" {
    region = "${local.aws_region}"
  }
  EOF
  }
```

Esse arquivo define:

- **Backend único** para todos os ambientes (S3 + DynamoDB).
- **Provider gerado automaticamente** (`provider.tf`), sem precisar duplicar.
- **Configurações padrão** centralizadas.

Ambiente que herda do raiz:

```
  # live/dev/terragrunt.hcl
  include "root" {
    path = find_in_parent_folders()
  }

  locals {
    env = "dev"
    tags = {
      Project = "curso-iac"
      Env     = "dev"
    }
  }

  inputs = {
    common_tags = local.tags
  }
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Aqui, o ***dev* herda tudo do raiz** e apenas define suas próprias variáveis (ex.: ***tags***). Se amanhã trocarmos a região AWS no arquivo raiz, todos os ambientes ficam sincronizados automaticamente.

Módulo dentro de um ambiente:

```
# live/dev/us-east-1/vpc/terragrunt.hcl

include "env" {
  path = find_in_parent_folders("terragrunt.hcl")
}

terraform {
  source = "../../../../modules/vpc"
}

inputs = {
  name       = "dev-vpc"
  cidr_block = "10.10.0.0/16"
  azs        = ["us-east-1a", "us-east-1b"]
}
```

Esse ***terragrunt.hcl* é filho do ambiente *dev***. Ele recebe automaticamente as configs do ***dev/terragrunt.hcl*** e do ***live/terragrunt.hcl***. Só precisa definir as entradas específicas para o módulo VPC.

O mesmo para **prod**, mudando apenas o necessário:

```
# infra/prod/us-east-1/vpc/terragrunt.hcl
include "env" {
  path = find_in_parent_folders("terragrunt.hcl")
}

terraform {
  source = "../../../../modules/vpc"
}

inputs = {
  name       = "prod-vpc"
  cidr_block = "10.20.0.0/16"
  azs        = ["us-east-1a","us-east-1b","us-east-1c"]
}
```

Fluxos de trabalho com Terragrunt (orquestração)

Além da organização, o Terragrunt facilita a **orquestração** de módulos.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Em projetos reais, aplicar “na ordem certa” é crítico (ex.: criar VPC antes do EKS). Terragrunt facilita com o uso de ***dependency e run-all***.

Dependências entre módulos (encadeamento de outputs):

```
# live/dev/us-east-1/eks/terragrunt.hcl

dependency "vpc" {
  config_path = "../vpc"
}

terraform {
  source = "../../../../modules/eks"
}

inputs = {
  cluster_name = "eks-dev"
  vpc_id       = dependency.vpc.outputs.vpc_id
  subnet_ids   = dependency.vpc.outputs.subnet_ids
}
```

Aqui, o EKS depende da VPC. O Terragrunt entende essa ordem automaticamente.

- Em ***plan/validate***, se o VPC ainda não existe, os mock_outputs quebram o ciclo localmente.
- Em **apply**, o Terragrunt busca os **outputs oficiais** do VPC primeiro.

Execução em cascata com run-all

No diretório ***live/dev/us-east-1***, podemos aplicar todos os módulos de uma só vez:

```
# Planejar todos os módulos abaixo (na ordem correta)
terragrunt run-all plan

# Aplicar todos os módulos (respeitando dependências)
terragrunt run-all apply

# Destruir todos (com confirmação)
terragrunt run-all destroy
```

O *run-all* executa **na ordem certa**, respeitando dependências. Isso é essencial em times grandes, onde não dá para aplicar módulo por módulo manualmente.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Centralização de backend

Como o backend foi definido no `live/terragrunt.hcl`, todos os ambientes usam o mesmo bucket S3 e tabela DynamoDB. Isso evita ter de copiar e colar o mesmo bloco em cada módulo:

```
    remote_state {
      backend = "s3"
      config = {
        bucket         = "iac-terraform-state"
        key            =
${path_relative_to_include()}/terraform.tfstate"
        region         = "us-east-1"
        dynamodb_table = "terraform-locks"
        encrypt        = true
      }
    }
```

Retentativas e ajustes de CLI:

```
    terraform {
      retryable_errors = [
        "Error acquiring the state lock",
        "RequestLimitExceeded",
      ]
      retry_max_attempts = 5
      retry_sleep_interval_sec = 10
      extra_arguments "common" {
        commands  = ["plan","apply","destroy"]
        arguments = ["-parallelism=10"]
      }
    }
```

Alternando Terraform <> OpenTofu sem mexer no código:

```
    # no raiz
    locals { tf_binary = "tofu" }     # ou "terraform"
    terraform_binary = local.tf_binary
```

Em times que estão migrando, esse truque evita “trocar” chamadas dentro dos módulos.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Boas práticas em times grandes

- **Pasta única de *`infra/`*** por organização/equipe; ambientes são pastas filhas. O ***terragrunt.hcl*** raiz com backend, provider e padrões; filhos só sobrescrevem o necessário.
- **Padronize nomes de diretórios**: ***`infra/<env>/<region>/<stack>/<módulo>`.***
- **Versione módulos**: `terraform { source = "git::https://...//modules/vpc?ref=v1.2.3" }` para evitar "surpresas".
- Use **`dependency`** sempre que um módulo precisa de outputs de outro (em vez de passar IDs manualmente).
- **CI/CD não-interativo**: `terragrunt run-all plan --terragrunt-non-interactive` e aplique approvals no pipeline.
- **Formatação**: `terragrunt hclfmt` no pre-commit/CI para manter os HCLs consistentes.
- **Segurança**: se for assumir roles, gere o `provider` via `generate` e mantenha as ARNs em `locals` ou secrets do pipeline.

Conclusão

Terragrunt não substitui Terraform/OpenTofu, ele os **amplifica**. Você mantém os módulos "puros", mas ganha **herança, uso do DRY, orchestration (*run-all*) e governança**. Em equipes grandes, isso é a diferença entre um monólito difícil de manter e uma base sustentável, previsível e pronta para escalar.

Em resumo:

- Elimina duplicação de código com **herança de configs**;
- Organiza ambientes de forma clara;
- Garante consistência com **remote state centralizado**;
- Orquestra execuções com **run-all**;
- Facilita a colaboração em times grandes.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Com ele, ambientes como **dev, staging e prod** passam a ser apenas **diferenciações de parâmetros**, e não cópias inteiras de infraestrutura. Isso significa menos esforço de manutenção, mais consistência e mais velocidade de entrega.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

O Terragrunt vem se consolidando como uma ferramenta de referência para **organizar múltiplos ambientes** e reduzir a complexidade de projetos de IaC. Um exemplo interessante é apresentado no blog da **env0**, uma plataforma SaaS que ajuda empresas a gerenciar e automatizar suas implantações de infraestrutura.

No artigo, os autores mostram como o Terragrunt permite que times **mantenham um repositório centralizado de módulos Terraform**, enquanto os ambientes reais (como dev*,* staging *e* prod*)* são definidos em diretórios separados, cada um com suas próprias variáveis. Esse padrão evita duplicação de código e dá mais clareza ao fluxo de trabalho, algo crítico em empresas que lidam com dezenas ou centenas de serviços em paralelo.

Além disso, o case discute como a integração do Terragrunt com pipelines de CI/CD e ferramentas de governança pode dar mais segurança ao ciclo de vida da infraestrutura. Isso inclui práticas como **execuções orquestradas com `run-all`**, uso de **dependências explícitas entre módulos** e até **hooks personalizados** para validar ou documentar a infraestrutura antes de aplicar as mudanças.

A grande lição aqui é que o Terragrunt não se limita a laboratórios acadêmicos, ele já é adotado por empresas e plataformas especializadas, como a env0, trazendo **escalabilidade, consistência e governança para a IaC em ambientes corporativos complexos**.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você conheceu o **Terragrunt** como uma camada que complementa o Terraform e o OpenTofu, trazendo mais organização e escalabilidade para projetos de IaC. Exploramos as principais **limitações do Terraform puro**, como a repetição de código entre ambientes, e vimos como o Terragrunt resolve esse problema com **herança de configurações**, **estrutura de diretórios clara** e **centralização de variáveis**.

Também analisamos exemplos práticos de como estruturar ambientes como **dev** e **prod**, aproveitando ao máximo o princípio **DRY (Don't Repeat Yourself)**. Você viu como o Terragrunt facilita o uso de **remote state centralizado**, permite definir dependências entre módulos e possibilita a execução de **pipelines orquestrados com o comando `run-all`**, garantindo que recursos sejam aplicados na ordem correta.

Por fim, entendemos como essas práticas podem ser aplicadas em times grandes, reduzindo riscos, aumentando a consistência e acelerando a entrega de infraestrutura confiável em múltiplos ambientes.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

CLOUDTHAT. **30% Faster Deployments and 99.9% Consistency with Terragrunt's Modular Architecture**. CloudThat, 2022. Disponível em: https://www.cloudthat.com/resources/case-study/30-faster-deployments-and-99-9-consistency-with-terragrunts-modular-architecture/. Acesso em: 21 out. 2025.

ENV0. **Terragrunt Tutorial: Examples and Use Cases**. env0 Blog, 2023. Disponível em: https://www.env0.com/blog/terragrunt. Acesso em: 21 out. 2025.

GRUNTWORK. **How to Manage Multiple Environments with Terraform Using Terragrunt.** Gruntwork Blog, 2020. Disponível em: https://www.gruntwork.io/blog/how-to-manage-multiple-environments-with-terraform-using-terragrunt/. Acesso em: 21 out. 2025.

GRUNTWORK. **Terragrunt Documentation**. Gruntwork Docs, 2025. Disponível em: https://terragrunt.gruntwork.io/docs/. Acesso em: 21 out. 2025.

HASHICORP. **Terraform Backend Types**. Terraform Documentation, 2025. Disponível em: https://developer.hashicorp.com/terraform/language/settings/backends. Acesso em: 21 out. 2025.

HASHICORP**. Terraform Remote State.** Terraform Documentation, 2025. Disponível em: https://developer.hashicorp.com/terraform/language/state/remote. Acesso em: 21 out. 2025.

KLIKA TECH. **Deployment Pipeline for Terragrunt Infrastructure**. Klika Tech Blog, 2022. Disponível em: https://klika-tech.com/blog/2022/04/10/deployment-pipeline-for-terragrunt-infrastructure/. Acesso em: 21 out. 2025.

SOFTSERVE. **How One Mount Group Expanded Into DevOps With SoftServe**. SoftServe Blog, 2021. Disponível em: https://www.softserveinc.com/en-us/resources/how-one-mount-group-expanded-into-devops. Acesso em: 21 out. 2025.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

terraform. github. terragrunt, automação.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com